# CPU Scheduling




By:

**Dr. H. K. Azad**

Assistant Professor (Sr)

SCOPE

Email: hiteshwarkumar.azad@vit.ac.in

# Outline

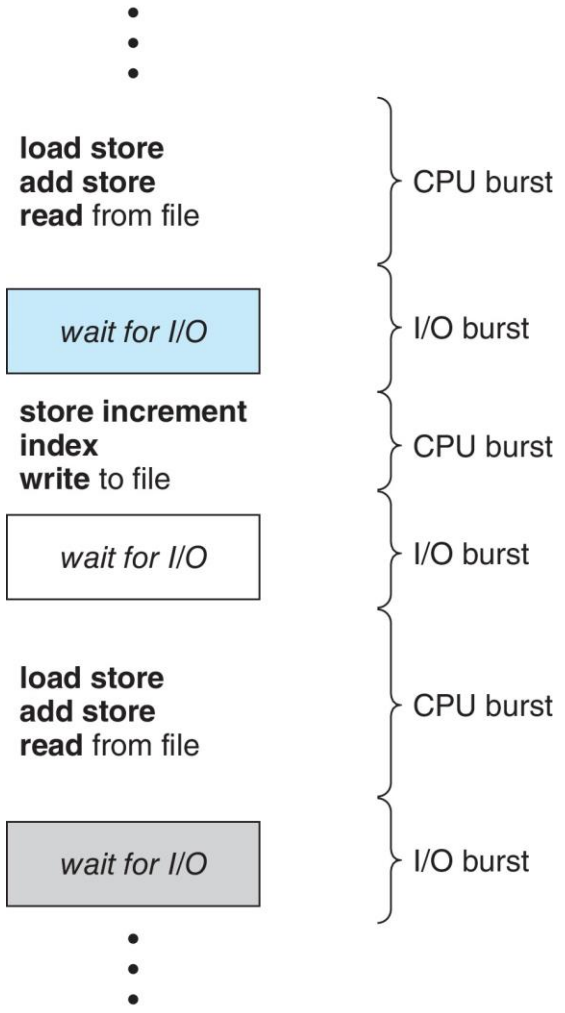
- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms

# Objectives

- Describe various CPU scheduling algorithms
- Assess CPU scheduling algorithms based on scheduling criteria
- Explain the issues related to multiprocessor and multicore scheduling
- Describe various real-time scheduling algorithms
- Describe the scheduling algorithms used in the Windows, Linux, and Solaris operating systems
- Apply modeling and simulations to evaluate CPU scheduling algorithms

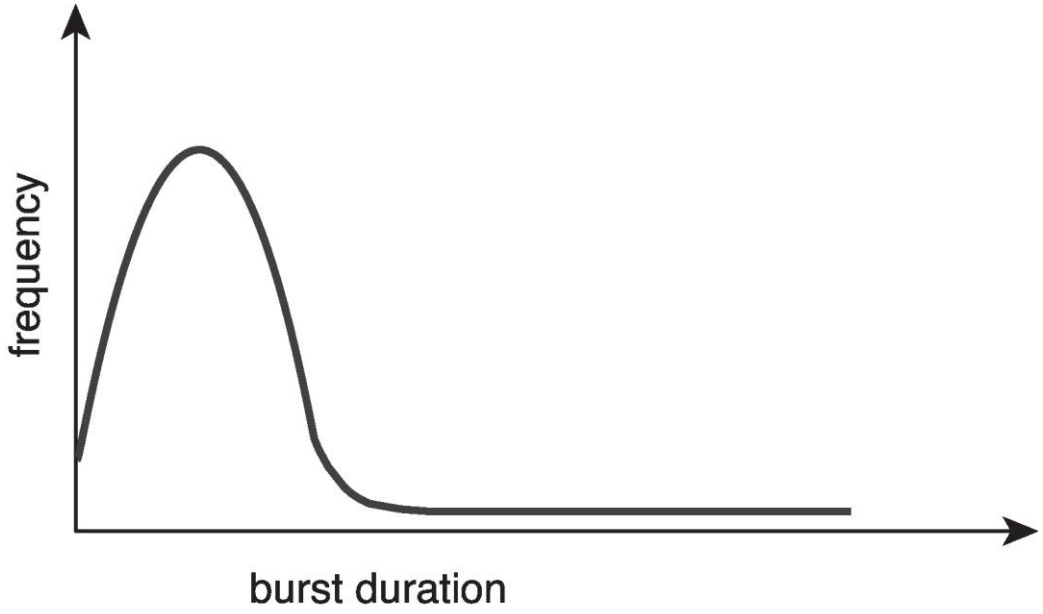
# Basic Concepts

- **Maximize CPU Utilization**: Achieved through **multiprogramming**.
- **CPU–I/O Burst Cycle**: Process execution alternates between:
  - **CPU Burst** → Execution on CPU (e.g., calculations, data processing)
  - **I/O Burst** → Waiting for I/O operations (e.g., file read/write)
- A **CPU burst** is always followed by an **I/O burst**.
- **Key focus**: Distribution of CPU bursts (short vs. long bursts).



# Histogram of CPU-burst Times

- **Observation**:
  - Most processes have **many short CPU bursts**.
  - Few processes have **longer CPU bursts**.
- **Reason**:
  - I/O-bound processes → Short CPU bursts.
  - CPU-bound processes → Long CPU bursts.



# CPU Scheduler

- The **CPU scheduler** selects from among the processes in ready queue, and allocates a CPU core to one of them
  - The ready queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates
- For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution.
- For situations 2 and 3, however, there is  a choice.

# Preemptive and Nonpreemptive Scheduling

- When scheduling takes place only under circumstances 1 and 4, the scheduling scheme is **nonpreemptive**.
- Otherwise, it is **preemptive**.
- Under Nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.
  - What is the potential problem?
- Virtually all modern operating systems including Windows, MacOS, Linux, and UNIX use preemptive scheduling algorithms.

# Preemptive Scheduling and Race Conditions

- Preemptive scheduling can result in race conditions when data are shared among several processes.
- Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, which are in an inconsistent state.

**Bank Account Race Condition:**

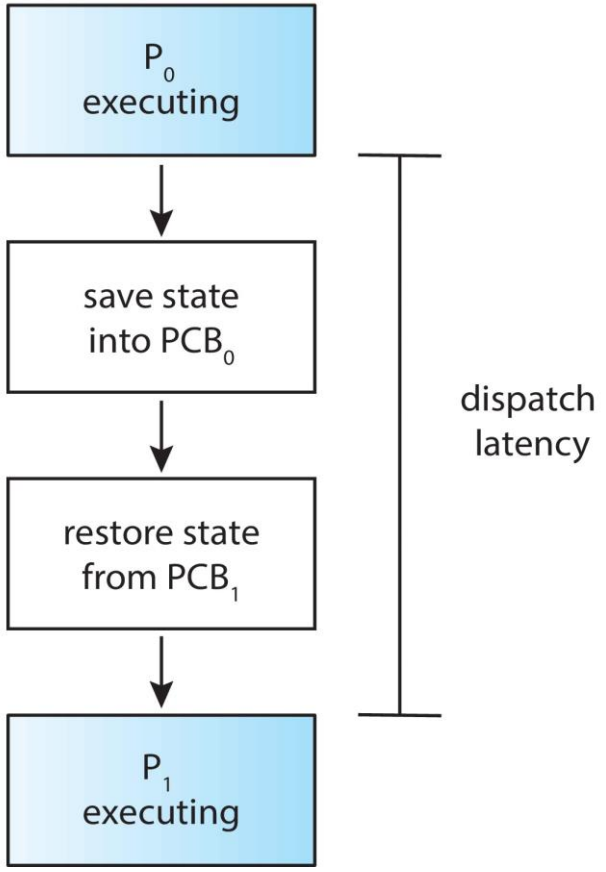
Imagine two ATM machines (threads) accessing the same **bank account balance** at the same time.
**Initial balance:** ₹100
Two people try to withdraw ₹70 at the same time.

Timeline of Events (Race Condition):

1. **ATM 1** checks: `balance >= 70` → `100 >= 70` → ✅
2. **ATM 2** checks: `balance >= 70` → `100 >= 70` → ✅
3. **ATM 1** subtracts 70 → balance becomes `30`.
4. **ATM 2** subtracts 70 → balance becomes `-40` ❌ (overdrawn).

# Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the CPU scheduler; this involves:
  - Switching context
  - Switching to user mode
  - Jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running



# Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
- **Throughput** – # of processes that complete their execution per time unit
- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced.

# Optimization Criteria for Scheduling Algorithms

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

# First- Come, First-Served (FCFS) Scheduling

- Processes are executed in the order they arrive in the ready queue.
- **Type**: **Non-preemptive** – once started, runs until completion.
- **Selection Rule**: Process with earliest arrival time is chosen first.
- **Tie-breaker**: If arrival times are equal, selection depends on implementation (e.g., lower PID).

✅ **Advantages:**

- Simple, easy to implement.
- Minimal context switching.
- Works well for processes with **long CPU burst times**.

⚠️ **Disadvantages:**

- Convoy effect: Short jobs wait for long jobs.
- Can result in high average waiting time.
- Not suitable for interactive or time-critical processes.

🔑 **Note:** Cannot be both preemptive & non-preemptive — FCFS is strictly **non-preemptive**.

# First- Come, First-Served (FCFS) Scheduling

- Example with 3 processes

| Process | Burst Time |
|---|---|
| $P_1$ | 24 |
| $P_2$ | 3 |
| $P_3$ | 3 |

- Suppose that the processes arrive in the order: $P_1$ , $P_2$ , $P_3$
  The Gantt Chart for the above schedule is:




- Waiting time for $P_1$ = 0; $P_2$ = 24; $P_3$ = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

# FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2 , P_3 , P_1$$

- The Gantt chart for the schedule is:




- Waiting time for $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case
- **Convoy effect** - short process behind long process
  - Consider one CPU-bound and many I/O-bound processes

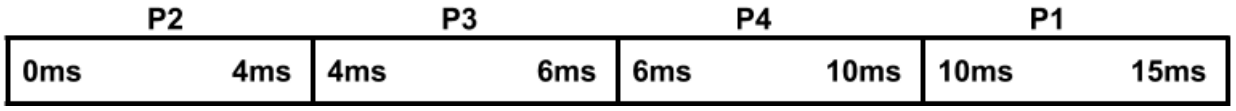
# Shortest-Job-First (SJF) Scheduling

- A scheduling algorithm that selects the process with the smallest estimated CPU burst time for execution next.
- **Process Selection**:
  - Associate each process with the length of its next CPU burst.
  - Schedule the process with the shortest burst time first.
- **Optimality**:
  - Minimizes the average waiting time for a given set of processes.
- **Type**:
  - Non-preemptive: Once a process starts execution, it runs until completion.

**Example**:

| Process | Arrival time | Burst time |
|---|---|---|
| P1 | 3 ms | 5 ms |
| P2 | 0 ms | 4 ms |
| P3 | 4 ms | 2 ms |
| P4 | 5 ms | 4 ms |

**Gantt Chart**



# Shortest-Job-First (SJF) Scheduling

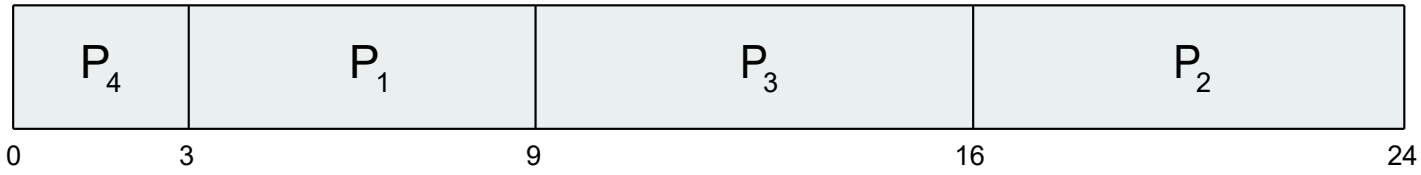
- **Advantages**
  - Efficient in terms of average waiting time.
  - Suitable for batch processing where burst times are known in advance.
- **Disadvantages**
  - **Starvation (Indefinite Blocking):**
    - Longer processes may suffer delays if shorter jobs keep arriving.
    - This can lead to processes waiting indefinitely.

- **Possible Solution to Starvation**
  - **Aging:** Gradually increase the priority of waiting processes so they eventually get scheduled.

# Example of SJF

| Process | Burst Time |
|---|---|
| $P_1$ | 6 |
| $P_2$ | 8 |
| $P_3$ | 7 |
| $P_4$ | 3 |

- SJF scheduling chart




- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

# Determining Length of Next CPU Burst

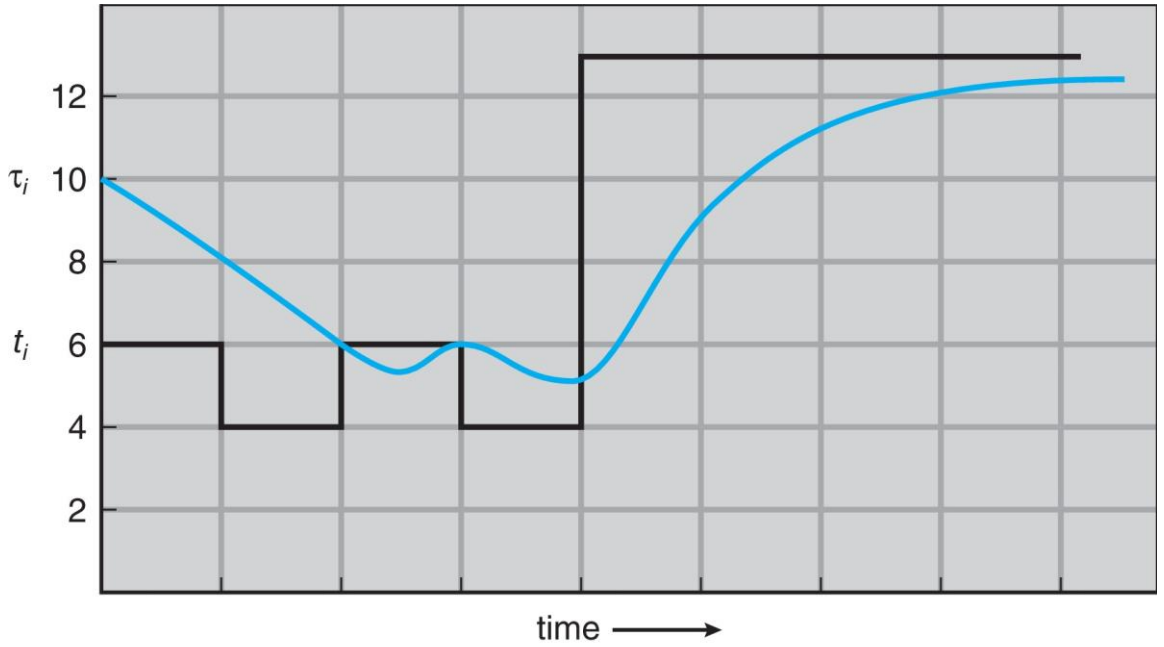
- Can only estimate the length – should be similar to the previous one
  - Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using exponential averaging

1. $t_n$ = actual length of $n^{th}$ CPU burst
2. $\tau_{n+1}$ = predicted value for the next CPU burst
3. $\alpha, 0 \le \alpha \le 1$
4. Define :

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\tau_n.$$

- Commonly, α set to ½

# Prediction of the Length of the Next CPU Burst




| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| CPU burst ($t_i$) | 6 | 4 | 6 | 4 | 13 | 13 | 13 | ... |
| "guess" ($\tau_i$) | 10 | 8 | 6 | 6 | 5 | 9 | 11 | 12 | ... |

# Examples of Exponential Averaging

- $\alpha = 0$
  - $\tau_{n+1} = \tau_n$
  - Recent history does not count
- $\alpha = 1$
  - $\tau_{n+1} = \alpha\, t_n$
  - Only the actual last CPU burst counts
- If we expand the formula, we get:

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\alpha\, t_{n-1} + \ldots$$
$$+ (1 - \alpha)^j \alpha\, t_{n-j} + \ldots$$
$$+ (1 - \alpha)^{n+1} \tau_0$$

- Since both $\alpha$ and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor

# Shortest Remaining Time First Scheduling

- A **preemptive** version of **Shortest Job Next (SJN)** scheduling where the process with the smallest **remaining** CPU burst time is always selected for execution.

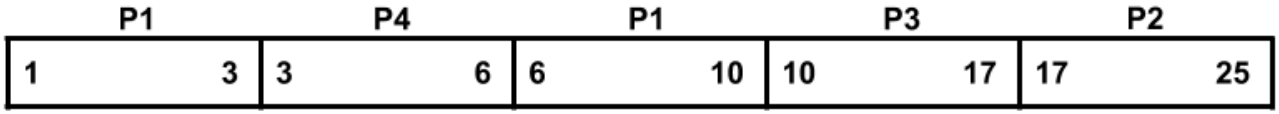
**How It Works:**

- When a **new process arrives**, the scheduler compares:
  - **Burst time of the new process**
  - **Remaining time of the current process**
- If the new process has a shorter remaining time, it **preempts** the current process.
- Continuously re-evaluates on each arrival or completion.

**Example:**

| Process | Arrival time | Burst time |
|---|---|---|
| P1 | 1 ms | 6 ms |
| P2 | 1 ms | 8 ms |
| P3 | 2 ms | 7 ms |
| P4 | 3 ms | 3 ms |

**Gantt Chart**



# Shortest Remaining Time First (SRTF) Scheduling

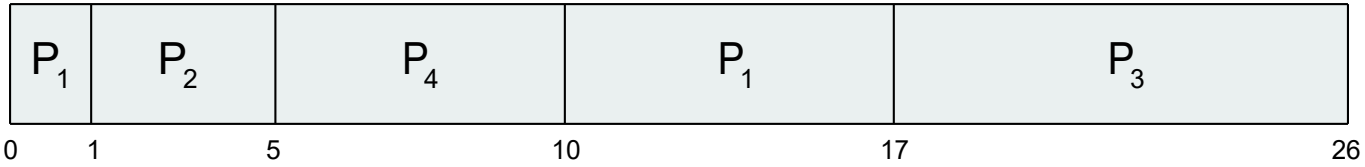
- **Advantages**
  - Handles **new arrivals** better than SJN, especially when short jobs appear while a long job is running.
  - Usually results in **lower average waiting time** compared to SJN when jobs arrive at different times.
- **Comparison with SJN**
  - **If all jobs arrive at time 0** → SJN and SRTF produce the **same results**.
  - **If jobs arrive at different times** → SRTF generally **outperforms SJN** in reducing waiting time.
- **Drawbacks**
  - **Higher scheduling overhead** due to frequent comparisons.
  - **Starvation risk** for long processes if short processes keep arriving.

# Example of Shortest-remaining-time-first

- Now we add the concepts of varying arrival times and preemption to the analysis

| Process | *Arrival* Time | Burst Time |
|---|---|---|
| $P_1$ | 0 | 8 |
| $P_2$ | 1 | 4 |
| $P_3$ | 2 | 9 |
| $P_4$ | 3 | 5 |

- *Preemptive* SJF Gantt Chart




- Average waiting time = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5
- Response Time = First CPU Start Time − Arrival Time

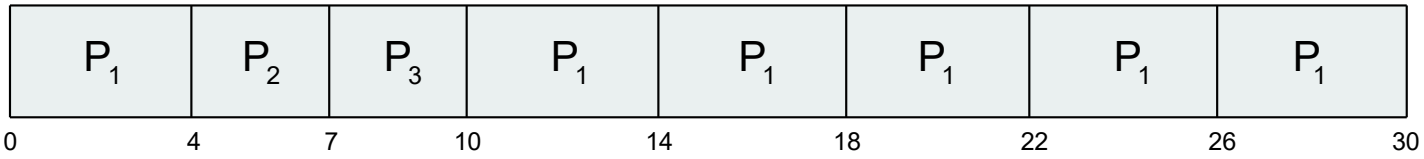
# Round Robin (RR)

- The round-robin (RR) scheduling algorithm is designed especially for time-sharing systems. It is similar to FCFS scheduling, but pre-emption is added to switch between processes.
- A small unit of time, called a time quantum or time slice. A time quantum is generally from 10 to 100 milliseconds. The ready queue is treated as a circular queue.
- Each process gets a small unit of CPU time (time quantum $q$). After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are $n$ processes in the ready queue and the time quantum is $q$, then each process gets $1/n$ of the CPU time in chunks of at most $q$ time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
  - $q$ large $\Rightarrow$ FIFO
  - $q$ small $\Rightarrow$ $q$ must be large with respect to context switch, otherwise overhead is too high

# Example of RR with Time Quantum = 4

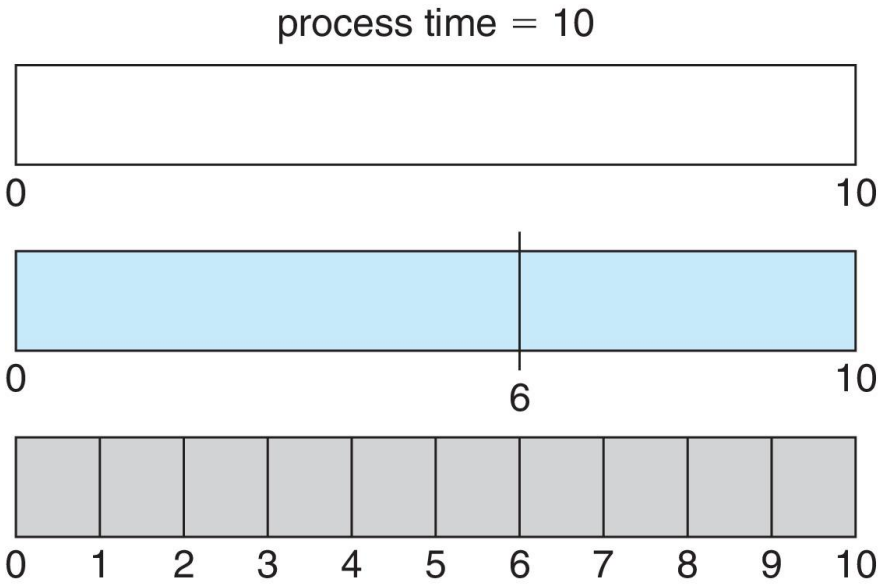
| Process | Burst Time |
|---|---|
| $P_1$ | 24 |
| $P_2$ | 3 |
| $P_3$ | 3 |

- The Gantt chart is:



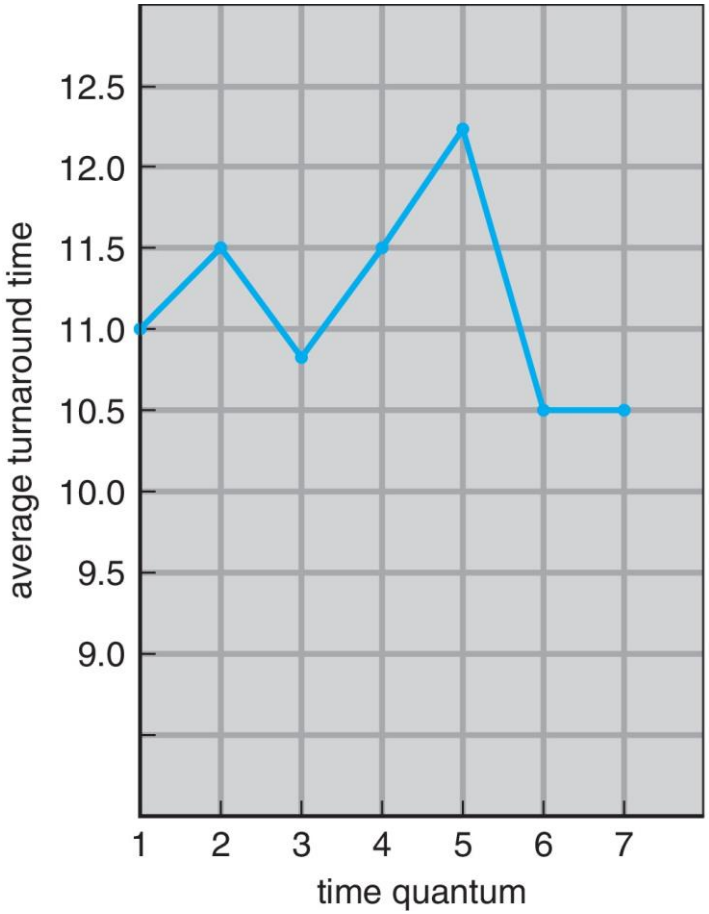

- Typically, higher average turnaround than SJF, but better ***response***
- q should be large compared to context switch time
  - q usually 10 milliseconds  to 100 milliseconds,
  - Context switch < 10 microseconds

# Time Quantum and Context Switch Time




| quantum | context switches |
|---|---|
| 12 | 0 |
| 6 | 1 |
| 1 | 9 |

# Turnaround Time Varies With The Time Quantum




| process | time |
|---|---|
| $P_1$ | 6 |
| $P_2$ | 3 |
| $P_3$ | 1 |
| $P_4$ | 7 |

80% of CPU bursts should be shorter than q

# Examples: MCQ

1. With round robin scheduling algorithm in a time shared system_____

   A. using very large time slices converts it into First come First served scheduling algorithm
   B. using very small time slices converts it into First come First served scheduling algorithm
   C. using extremely small time slices increases performance
   D. using very small time slices converts it into Shortest Job First algorithm

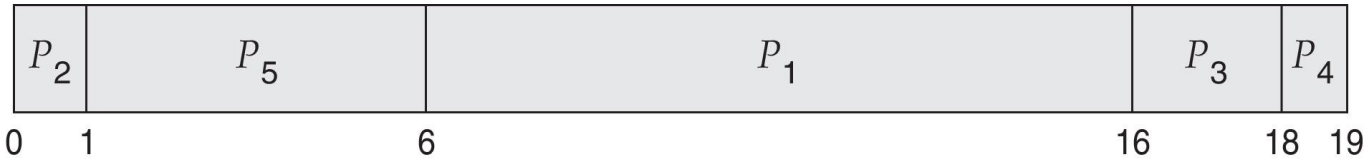
# Priority Scheduling

- **Definition:** Each process is assigned a **priority number** (integer).
  - Smaller number → **Higher priority**
  - Larger number → **Lower priority**
- **How it works:**
  The CPU is allocated to the process with the **highest priority**.
- **Types:**
  - **Preemptive** – If a new process arrives with a higher priority, it preempts the current process.
  - **Non-preemptive** – The currently running process continues until completion, even if a higher-priority process arrives.
- **Problem: Starvation**
  - Low-priority processes might never get CPU time if higher-priority processes keep arriving.
- **Solution: Aging**
  - Gradually increase the priority of waiting processes over time to ensure eventual execution.
- **Note:**
  Shortest Job First (SJF) is a special case of priority scheduling where **priority is the inverse of the predicted next CPU burst time**.

# Example of Priority Scheduling

| Process | Burst Time | Priority |
|---|---|---|
| $P_1$ | 10 | 3 |
| $P_2$ | 1 | 1 |
| $P_3$ | 2 | 4 |
| $P_4$ | 1 | 5 |
| $P_5$ | 5 | 2 |

- Priority scheduling Gantt Chart




- Average waiting time = 8.2




**Round robin scheduling falls under the category of . . .**

A) Non-preemptive scheduling

C) Both

B) None

D) Preemptive scheduling

Next

**Which of the following scheduling algorithms is non-preemptive?**

A) Preemptive Priority

C) Round Robin

B) Shortest Remaining Time First (SRTF)

D) FCFS

Next

**The “convoy effect” in CPU scheduling happens when:**

A) A short process executes before a long process

C) All processes arrive at the same time

B) CPU remains idle for a long time

D) A long process delays the execution of short processes

Next

**Which of the following scheduling algorithms can be both preemptive and non-preemptive?**

A) Round Robin

C) FCFS

B) Multilevel Queue

D) SJF

Next

**Which scheduling algorithm is most prone to starvation?**

A) FCFS

C) Round Robin

B) Non-preemptive Priority Scheduling

D) Preemptive Priority Scheduling

Next

**Starvation can be avoided by:**

A) Using FCFS scheduling

C) Aging

B) Reducing quantum time

D) Using shortest job first always

Well done!

# Example: GATE 2017

Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority (0 is the highest priority) shown below. None of the processes have I/O burst time.

| Process | Arrival Time | Burst Time | Priority |
|---|---|---|---|
| $P_1$ | 0 | 11 | 2 |
| $P_2$ | 5 | 28 | 0 |
| $P_3$ | 12 | 2 | 3 |
| $P_4$ | 2 | 10 | 1 |
| $P_5$ | 9 | 16 | 4 |

The average waiting time (in milliseconds) of all the processes using preemptive priority scheduling algorithm is ____.

**Note:** This question appeared as Numerical Answer Type.

**(A)** 29

**(B)** 30

**(C)** 31

**(D)** 32

# Priority Scheduling w/ Round-Robin

- Run the process with the highest priority. Processes with the same priority run round-robin
- Example:

| Process | Burst Time | Priority |
|---|---|---|
| $P_1$ | 4 | 3 |
| $P_2$ | 5 | 2 |
| $P_3$ | 8 | 2 |
| $P_4$ | 7 | 1 |
| $P_5$ | 3 | 3 |

- Gantt Chart with time quantum = 2



# MCQ

Q1. Which of the following statements are true? (GATE 2010)

I. Shortest remaining time first scheduling may cause starvation

II. Preemptive scheduling may cause starvation

III. Round robin is better than FCFS in terms of response time

a) only

b) I and III only

c) II and III only

d) I, II and III

Q2. Which of the following process scheduling algorithm may lead to starvation

a) FIFO

b) Round Robin

c) Shortest Job Next

d) None

Q3. Which of the following scheduling algorithms may cause starvation ?

a. First-come-first-served b. Round Robin

c. Priority d. Shortest process next

e. Shortest remaining time first

(A) a, c and e

(B) c, d and e

(C) b, d and e

(D) b, c and d

# Example:1

Consider the 3 processes, P1, P2 and P3 shown in the table.

| Process | Arrival time | Time Units Required |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 7 |
| P3 | 3 | 4 |

The completion order of the 3 processes under the policies FCFS and RR2 (round robin scheduling with CPU quantum of 2 time units) are

**(A)**

FCFS: P1, P2, P3
RR2: P1, P2, P3

**(B)**

FCFS: P1, P3, P2
RR2: P1, P3, P2

**(C)**

FCFS: P1, P2, P3
RR2: P1, P3, P2

**(D)**

FCFS: P1, P3, P2
RR2: P1, P2, P3

# Example:2

An operating system uses shortest remaining time first scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in milliseconds):

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 12 |
| P2 | 2 | 4 |
| P3 | 3 | 6 |
| P4 | 8 | 5 |

The average waiting time (in milliseconds) of the processes is _____.

**(A)** 4.5

**(B)** 5.0

**(C)** 5.5

**(D)** 6.5

# Example:3

Consider the following CPU processes with arrival times (in milliseconds) and length of CPU bursts (in milliseconds) except for process P4 as given below:

| Process | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 5 |
| P2 | 1 | 1 |
| P3 | 3 | 3 |
| P4 | 4 | x |

If the average waiting time across all processes is 2 milliseconds and pre-emptive shortest remaining time first scheduling algorithm is used to schedule the processes, then find the value of x ?

**(A)** 1

**(B)** 2

**(C)** 4

**(D)** None of these

# Example:4

Four jobs to be executed on a single processor system arrive at time 0 in the order A, B, C, D. Their burst CPU time requirements are 4, 1, 8, 1 time units respectively. The completion time of A under round robin scheduling with time slice of one time unit is

**(A)** 10

**(B)** 4

**(C)** 8

**(D)** 9

# Example-5

Q: Consider three CPU-intensive processes, which require 10, 20 and 30 time units and arrive at times 0, 2 and 6, respectively. How many context switches are needed if the operating system implements a shortest remaining time first scheduling algorithm? Do not count the context switches at time zero and at the end.

a) 1

b) 2

c) 3

d) 4

# Examples

Consider the set of four processes P1, P2, P3, and P4, each with a process execution time of 5, 10, 15, and 20 milliseconds. The process execution consists of 80% CPU burst time and 20% I/O burst time. If the CPU scheduling policy is round robin with time stamp 2 milliseconds. What will be the average turnaround time.

Consider three processes, all arriving at time zero, with total execution time of 10, 20 and 30 units, respectively. Each process spends the first 20% of execution time doing I/O, the next 70% of time doing computation, and the last 10% of time doing I/O again. The operating system uses a shortest remaining compute time first scheduling algorithm and schedules a new process either when the running process gets blocked on I/O or when the running process finishes its compute burst. Assume that all I/O operations can be overlapped as much as possible. For what percentage of time does the CPU remain idle?

(A) 0% (B) 10.6% (C) 30.0% (D) 89.4%

| I | P1 | P2 | P3 | I |
0 2 9 23 44 47

# Example

Suppose that processes P1, P2, . . ., P5 arrive for execution at the times given in table. Each process will run for the amount of total time listed, which includes CPU and I/O burst time. Each process firstly spends 25% of time in handling I/O and remaining time uses CPU cycle.

| Process ID | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 4 |
| P2 | 1 | 16 |
| P3 | 10 | 20 |
| P4 | 5 | 8 |
| P5 | 4 | 24 |

FCFS:

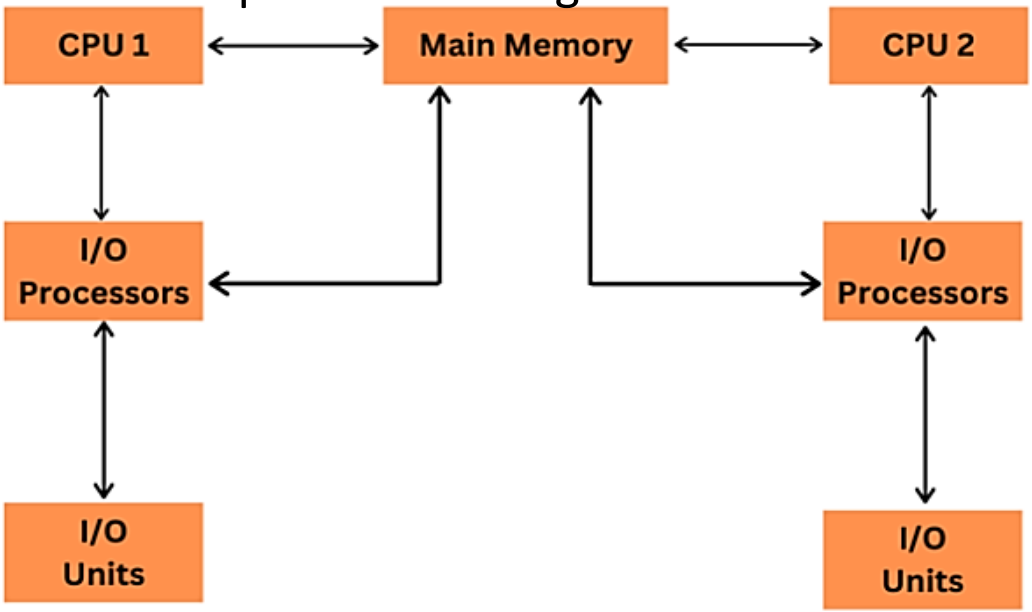
| I | P1 | I | P2 | P4 | P5 | P3 |
0 1 4 5 17 23 41 56

SJF(Pre-emptive):

| I | P1 | I | P2 | P4 | P2 | P3 | P5 |
0 1 4 5 8 14 23 38 56

# Multiprocessor scheduling

- **Definition:** Multiprocessor scheduling focuses on designing scheduling policies for systems with more than one processor or core.
- **Goal:** Maximize CPU utilization, throughput, and system performance while minimizing waiting and turnaround times.
- **Need for Multiprocessors:**
  - Modern systems often use multiple processors/cores for parallelism.
  - Improves performance for computation-heavy workloads.
- **Key Tasks:**
  - Assign processes to processors/cores.
  - Decide how long a process should run before switching.
  - Balance load to avoid processor idling.



# Multiprocessor scheduling

**Challenges:**

- Avoiding load imbalance.
- Minimizing inter-processor communication overhead.
- Handling synchronization and shared resources.

**Objective:** Ensure fairness and priority adherence while efficiently using all available processors.

**Scenario:**

We have **two processors** (**CPU1** and **CPU2**) and **five processes** with given burst times.

We will use **load balancing with FCFS** inside each CPU.

**Step 1 – Assign Processes to CPUs**

We aim to balance total execution time between CPUs:

- **CPU1:** P1 (10 ms), P4 (6 ms), P5 (2 ms) → Total = **18 ms**
- **CPU2:** P2 (5 ms), P3 (8 ms) → Total = **13 ms**

| Process | Burst Time (ms) | Arrival Time (ms) |
|---|---|---|
| P1 | 10 | 0 |
| P2 | 5 | 0 |
| P3 | 8 | 0 |
| P4 | 6 | 0 |
| P5 | 2 | 0 |

**Step 2 – Execution Timeline**

Since both CPUs run **in parallel**:

CPU1 (18 ms total) : P1 (0–10) → P4 (10–16) → P5 (16–18)

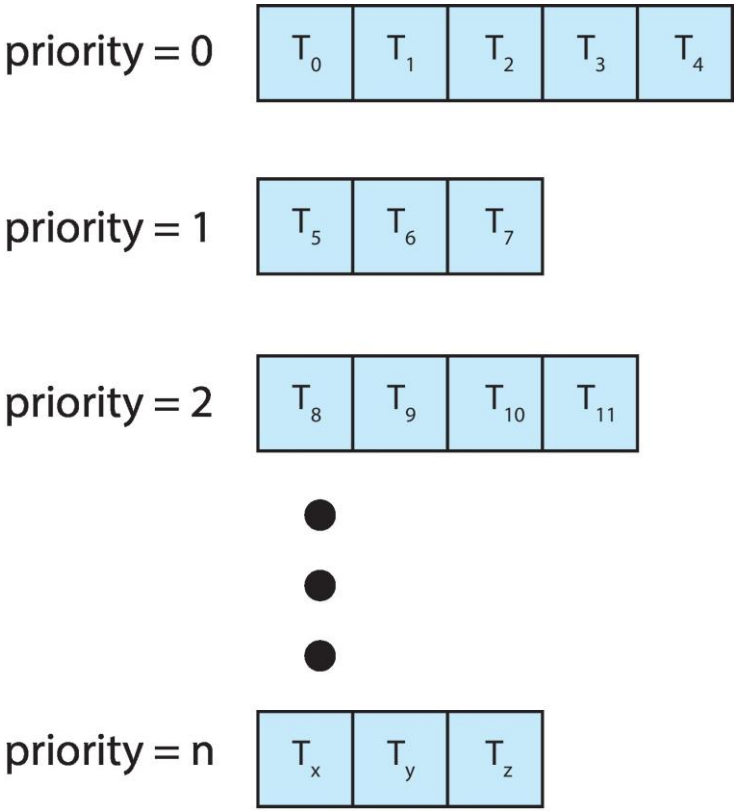
CPU2 (13 ms total) : P2 (0–5) → P3 (5–13)

**Step 3 – Completion Times**

- **CPU1 finishes at 18 ms & CPU2 finishes at 13 ms**
- **Overall completion time (makespan) = max(18, 13) = 18 ms**

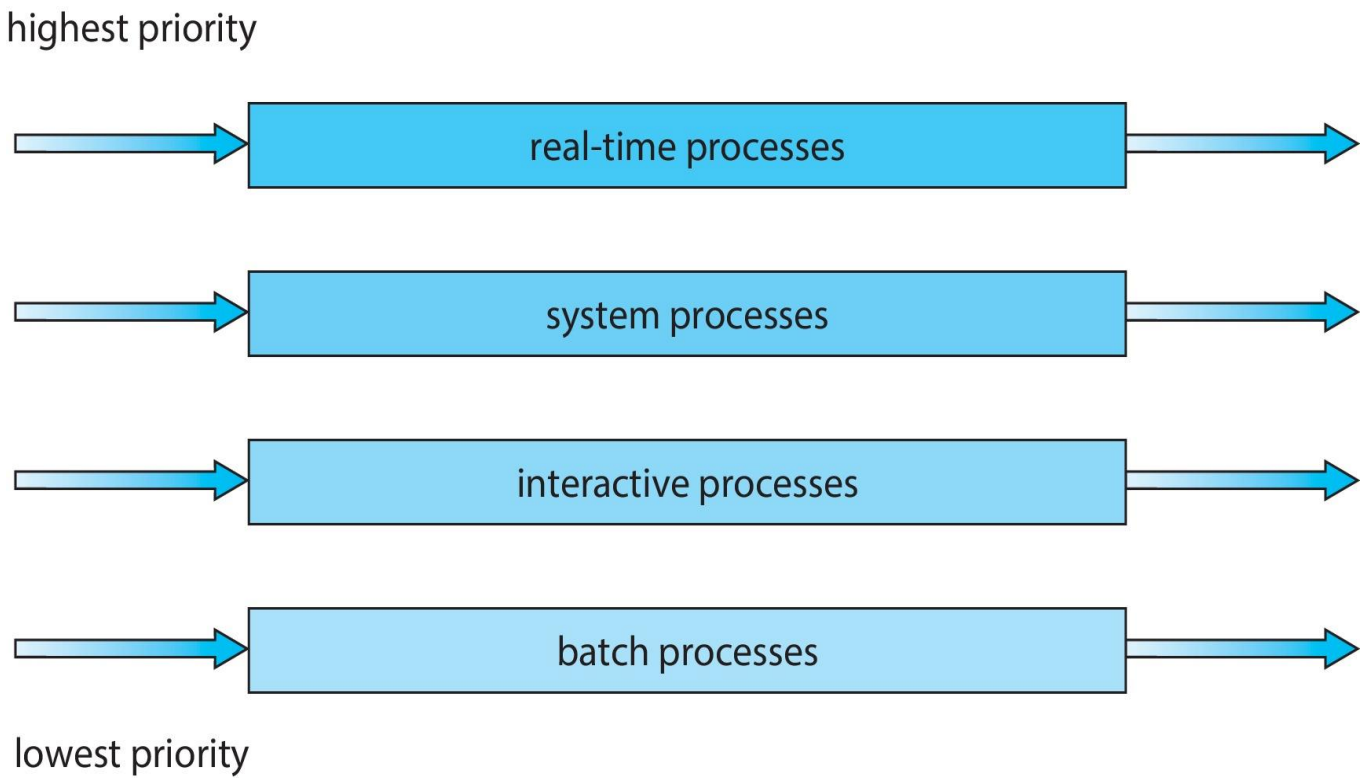
# Multilevel Queue

- The ready queue consists of multiple queues
- Example:
  - Priority scheduling, where each priority has its separate queue.
  - Schedule the process in the highest-priority queue!



# Multilevel Queue

- Prioritization based upon process type



# Multilevel Feedback Queue

- A process can move between the various queues.
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - Number of queues
  - Scheduling algorithms for each queue
  - Method used to determine when to upgrade a process
  - Method used to determine when to demote a process
  - Method used to determine which queue a process will enter when that process needs service
- Aging can be implemented using multilevel feedback queue

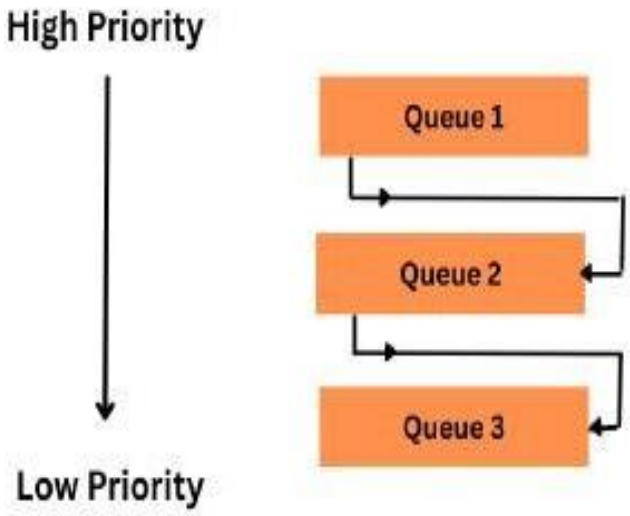
# Multilevel Feedback Queue Scheduling

## Definition:

A CPU scheduling algorithm that uses multiple queues with different priority levels, allowing processes to move between queues based on their behavior and execution history.

## Key Features:

- Multiple queues, each with its own **priority** and **time quantum**.
- Processes start in a high-priority queue and move down if they use up their time slice.
- Short jobs get executed quickly; long jobs gradually move to lower-priority queues.
- Can prevent **starvation** by periodically boosting priorities.



# Multilevel Feedback Queue (MLFQ) Scheduling

## 🔷 Working Principle:

- **New processes** enter the highest-priority queue.
- **Scheduling within a queue:** Usually **Round Robin** or **FCFS**.
- If a process uses its full quantum, it is **demoted** to a lower-priority queue.
- If a process waits too long, it may be **promoted** to avoid starvation.

## 🔷 Advantages:

✅ Good balance between responsiveness and throughput.
✅ Adapts to different types of processes automatically.

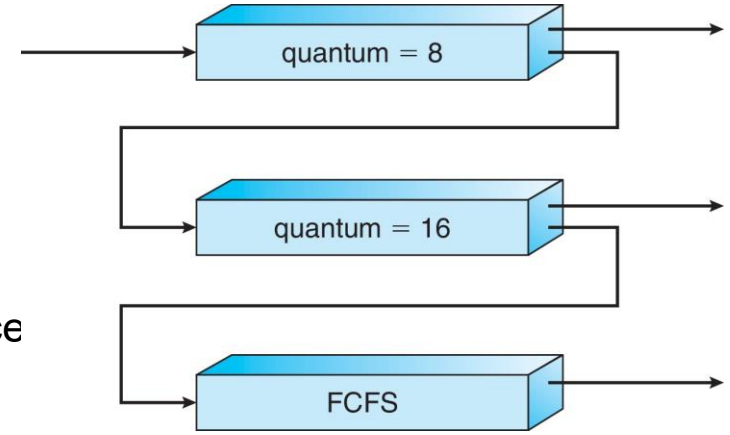
## 🔷 Disadvantages:

⚠ Complex to implement and tune.
⚠ Priority inversion possible if not managed properly.

**High Priority Q1 → Short Quantum → Interactive tasks**
↓
**Medium Priority Q2 → Medium Quantum → Mixed tasks**
↓
**Low Priority Q3 → Long Quantum → CPU-bound tasks**

# Example of Multilevel Feedback Queue

- Three queues:
  - $Q_0$ – RR with time quantum 8 milliseconds
  - $Q_1$ – RR time quantum 16 milliseconds
  - $Q_2$ – FCFS
- Scheduling
  - A new process enters queue $Q_0$ which is served in RR
    - When it gains CPU, the process receives 8 milliseconds
    - If it does not finish in 8 milliseconds, the proce is moved to queue $Q_1$
  - At $Q_1$ job is again served in RR and receives 16 additional milliseconds
    - If it still does not complete, it is preempted and moved to queue $Q_2$



# MLFQ Scheduling: Example

**Assumptions**

- **Three queues:**
  - **Q1** → Round Robin (Quantum = 4 ms) — highest priority
  - **Q2** → Round Robin (Quantum = 8 ms) — medium priority
  - **Q3** → FCFS — lowest priority
- **Rules:**
  - A new process starts in Q1.
  - If it uses up its time quantum without finishing, it moves down one queue.
  - If it finishes before the quantum expires, it stays at its priority level.
  - Higher-priority queues preempt lower ones.

Process Data:

| Process | Arrival Time (ms) | Burst Time (ms) |
|---|---|---|
| P1 | 0 | 10 |
| P2 | 1 | 4 |
| P3 | 2 | 6 |
| P4 | 3 | 8 |

# MLFQ Scheduling: Example

**Scheduling Steps:**

**Step 1 — Start in Q1 (Quantum = 4)**

- **Time 0–4** → P1 runs for 4 ms, remaining = 6 ms → moves to Q2.
- **Time 4–8** → P2 runs for 4 ms, finishes (burst exactly = quantum).
- **Time 8–12** → P3 runs for 4 ms, remaining = 2 ms → moves to Q2.
- **Time 12–16** → P4 runs for 4 ms, remaining = 4 ms → moves to Q2.

**Step 2 — Q1 is empty, go to Q2 (Quantum = 8)**

- Q2 now has: P1 (6 ms), P3 (2ms), P4 (4 ms).
- **Time 16–22** → P1 runs for 6 ms, finishes.
- **Time 22–24** → P3 runs for 2 ms, finishes
- **Time 24–28** → P4 runs for 4 ms, finishes.

Gantt Chart:

```
0   4   8   12  16  22  24  28
|P1 |P2 |P3 |P4 |P1 |P3 |P4 |
```

# Multilevel Queue Scheduling

- **Definition:** Scheduling technique where **ready processes are divided into multiple queues** based on priority or type.
- **Goal:** Provide different scheduling policies for different types of processes.
- **Key Characteristics:**
  - Queues are fixed (e.g., system processes, interactive jobs, batch jobs).
  - Each queue may use a **different scheduling algorithm** (e.g., Round Robin for interactive, FCFS for batch).
  - No movement between queues (unlike Multilevel Feedback Queue Scheduling).
  - CPU time is allocated to each queue according to a fixed priority or time slice

# Example

## Scenario

- We have **one CPU** but **two queues** based on process type:
- **Queue 1 (High Priority)** – Interactive processes
  - **Scheduling Policy:** Round Robin (Time Quantum = 4 ms)
- **Queue 2 (Low Priority)** – Batch processes
  - **Scheduling Policy:** First Come First Serve (FCFS)
- **Important Rule:**
  CPU always serves **Queue 1 first**. Queue 2 only runs when Queue 1 is empty.

Final Gantt Chart:

| P1 | P2 | P1 | P2 | P3 | P4 |
0 4 8 10 14 21 26

| Process | Queue | Burst Time (ms) | Arrival Time (ms) |
|---|---|---|---|
| P1 | Q1 | 6 | 0 |
| P2 | Q1 | 8 | 0 |
| P3 | Q2 | 7 | 0 |
| P4 | Q2 | 5 | 0 |

# Examples

**Consider a system that uses Multilevel Queue Scheduling with two queues: Queue 1: High-priority queue using Pre-emptive Shortest Job First scheduling. Queue 2: Low-priority queue using FCFS scheduling. Given the following set of processes with their arrival times, burst times, and queue assignments, draw the Gantt chart and calculate the average turnaround time and average waiting time.**

| Process ID | Arrival Time | Burst Time | Queue No. |
|---|---|---|---|
| P1 | 0 | 6 | 2 |
| P2 | 2 | 2 | 1 |
| P3 | 5 | 1 | 2 |
| P4 | 6 | 5 | 2 |
| P5 | 7 | 2 | 1 |
| P6 | 8 | 4 | 2 |
| P7 | 9 | 8 | 1 |
| P8 | 11 | 3 | 2 |
| P9 | 14 | 2 | 1 |

```
| P1 | P2 | P1 | P5 | P7 | P9 | P7 | P1 | P3 | P4 | P6 | P8 |
0    2    4    7    9    14   16   19   20   21   26   30   33
```

A computer system uses Multilevel Queue CPU Scheduling with three priority queues. The queues are arranged in descending order of priority, where a lower queue number indicates higher priority. Each queue follows a different CPU scheduling algorithm. Processes are permanently assigned to a queue and cannot move between queues. The scheduler always selects a process from the highest priority non-empty queue, and scheduling is preemptive between queues.

The scheduling policies for the queues are as follows:

- Queue Q1 uses Preemptive Priority Scheduling
- Queue Q2 uses First Come First Served (FCFS)
- Queue Q3 uses Round Robin Scheduling with a time quantum of 3 milliseconds

The details of the processes are given in the table below.

| Process ID | Arrival Time (ms) | Burst Time (ms) | Priority | Queue |
|---|---|---|---|---|
| P1 | 0 | 10 | 2 | Q2 |
| P2 | 1 | 6 | 1 | Q1 |
| P3 | 2 | 8 | 3 | Q3 |
| P4 | 3 | 4 | 1 | Q1 |
| P5 | 4 | 5 | 2 | Q2 |
| P6 | 4 | 9 | 2 | Q1 |
| P7 | 5 | 13 | 4 | Q3 |
| P8 | 5 | 4 | 1 | Q2 |
| P9 | 7 | 1 | 5 | Q3 |

a) Draw the Gantt chart for the given set of processes using Multilevel Queue Scheduling. (4 Marks)

b) Calculate the Waiting Time, Turnaround Time, and Response Time for each process. (3 Marks)

c) Determine the number of context switches that occur during execution and explain their causes. (3 Marks)

Consider a system that uses Multilevel Queue Scheduling with following criteria.

- System has three queues (Q1, Q2 and Q3) and priority decreases from Q1 to Q3
- Q1: Priority Scheduling (Non pre-emptive, lower number = higher priority)
- Q2: Round Robin (time quantum = 4)
- Q3: FCFS
- Higher-priority queues can preempt lower-priority queues
- CPU remains idle if no process is ready

| Process | Priority | Burst Time (ms) | Arrival Time (ms) | Queue |
|---|---|---|---|---|
| P1 | 3 | 6 | 0 | Q3 |
| P2 | 2 | 4 | 1 | Q1 |
| P3 | 1 | 3 | 2 | Q2 |
| P4 | 6 | 2 | 15 | Q1 |
| P5 | 8 | 5 | 16 | Q3 |
| P6 | 2 | 7 | 17 | Q2 |
| P7 | 1 | 3 | 20 | Q1 |

Draw the Gantt chart and calculate the average turnaround time, average waiting time, and average response time.

# Example

A system uses **non-preemptive and preemptive scheduling policies** depending on process priority. The following processes arrive for execution:

| Process | Arrival Time (ms) | Burst Time (ms) | Priority (1 = highest) | I/O Start (ms from arrival) | I/O Duration (ms) |
|---|---|---|---|---|---|
| P1 | 0 | 12 | 2 | 4 | 3 |
| P2 | 1 | 8 | 1 | — | — |
| P3 | 2 | 14 | 3 | 6 | 4 |
| P4 | 3 | 6 | 2 | — | — |

**Additional details:**

- Context switching time: **1 ms**
- Preemptive Priority Scheduling: If a higher priority process arrives, it **preempts** the current one.
- If two processes have the same priority, **Shortest Remaining Time First (SRTF)** is used.
- After I/O completion, the process is ready again and considered for CPU based on the same priority rules.
- Assume I/O is handled in parallel and doesn't block CPU scheduling.
- Waiting time is calculated excluding I/O delays.

# Example

A university computer system runs different types of tasks. The operating system uses a Multi-Level Queue Scheduling algorithm with two queues to schedule the tasks:

i. Foreground Queue (Interactive Processes) – uses Round Robin scheduling with time quantum = 4 ms.
ii. Background Queue (Batch Processes) – uses FCFS scheduling.

The foreground queue has higher priority than the background queue.

The processes are given below:

| Process | Type | Arrival Time | Burst Time |
|---|---|---|---|
| P1 | Foreground | 0 | 10 |
| P2 | Foreground | 1 | 6 |
| P3 | Background | 0 | 8 |
| P4 | Background | 2 | 5 |

Draw the Gantt Chart using Multi-Level Queue Scheduling and calculate the average waiting time.

| P1 | P2 | P1 | P2 | P1 | P3 | P4 |
0 4 8 12 14 16 24 29

Average Waiting Time = 12.75 ms
Average Turnaround Time = 20 ms

4. Suppose that processes P1, P2, . . ., P5 arrive for execution at the times given in table. Each process will run for the amount of total time listed, which includes CPU and I/O burst time. Each process spends 25% of time in handling I/O and during remaining time uses CPU cycle.

| Process ID | Arrival Time | Burst Time |
|---|---|---|
| P1 | 0 | 4 |
| P2 | 1 | 16 |
| P3 | 10 | 20 |
| P4 | 5 | 8 |
| P5 | 4 | 24 |

Draw the Gantt chart and calculate the average waiting time, average turn around time and response time of each process while scheduling is done using following algorithms

(i) FCFS
(ii) SJF ( non Pre-emptive)
(iii) SJF (Pre-emptive)

10 CC

**ANS :**

(i) FCFS ( 3 Marks)
(ii) SJF ( non Pre-emptive) (3 Marks)
(iii) SJF (Pre-emptive) (4 marks)

The I/O and CPU Burst Time Separated :

| Process ID | Arrival Time | I/O | CPU |
|---|---|---|---|
| P1 | 0 | 1 | 3 |
| P2 | 1 | 4 | 12 |
| P3 | 10 | 5 | 15 |
| P4 | 5 | 2 | 6 |
| P5 | 4 | 6 | 18 |

# End